Day13 - 模块与包

- [x] 掌握模块
  - [x] 模块的导入方式
  - [x] 自定义模块
- [x] 掌握包
  - [x] 如何自定义包
  - [x] 安装第三方包（py最强大的地方）

一、模块

1、什么是模块

一个模块就是一个Python文件，以.py 结尾,能定义函数,类和变量,也能包含可执行的代码

我们可以认为不同的模块就是不同工具包,每一个工具包中都有各种不同的工具(如函数)供我们使用进而实现各种不同的功能。

2、 模块的导入

模块在使用之前需要先导入正在开发的文件

导入语法:

```
[from 模块名] import [类|变量|函数|*] [as 别名]
# *表示导入所有
```

常用的组合形式如:

1. `import 模块名`
2. `from 模块名 import 类、变量、方法等`
3. `from 模块名 import *`
4. `import 模块名 as 别名`
5. `from 模块名 import 功能名 as 别名`

1、用法一：导模块

导入模块，基本语法:

```
# 导入
import 模块名
import 模块名1，模块名2

# 使用
模块名.功能名()
```

案例：使用示例

```python
# 导入时间模块
import time
print("开始") # 打印 开始
# 使用time模块中睡眠功能(其中还有众多其他功能)
# 可以让程序睡眠10秒后再继续执行
time.sleep(10)
print("结束") # 十秒后打印结束
```

2、用法二：导功能

导入模块中的一个功能，基本语法

```python
# 导入
from 模块名 import 功能名
# 使用
功能名()
```

案例：使用示例

```python
# 导入时间模块中的sleep方法
# 只能使用time模块中导入的sleep的方法
from time import sleep
print("开始") # 打印 开始
# 让程序睡眠10秒后再继续执行
sleep(10)
print("结束") #十秒后打印 结束
```

3、用法三：别名

我们在导入模块或功能时，可以给它们取一个简要的别名，基本语法

```python
# 导入一：模块定义别名
import 模块名 as 别名
# 使用一：
别名 .功能名()

# 导入二：功能定义别名
from 模块名 import 功能 as 别名
# 使用二：
别名()
```

- 使用示例一:

```python
# 本名time将不可用
import time as tt
print("开始") # 打印 开始
# 让程序睡眠10秒后再继续执行
# 通过别名调用
tt.sleep(10)
print("结束") #十秒后打印 结束
```

- 使用示例二:

```python
# 本名sleep将不可用
from time import sleep as sl
print("开始") # 打印 开始
# 让程序睡眠10秒后再继续执行
sl(10)
print("结束") # 十秒后打印 结束
```

4、用法四：通配符 *

使用通配符* 可以导入一个模块中的所有成员。基本语法

```python
# 导入
from 模块名 import *

# 使用
功能名()
```

- 使用示例:

```python
# 导入时间模块中的全部功能
# 导入效果与(2.1)一致 ,使用与(2.2)一致
from time import *
print("开始") # 打印 开始
# 让程序睡眠10秒后再继续执行
sleep(10)
print("结束") #十秒后打印 结束
```

总结

- from可以省略，直接import
- as别名可以省略
- 通过”.”（理解为xx的）来调用模块提供的功能
- 模块的导入一般写在代码文件的开头位置

3、自定义模块

Python中已经帮我们实现了很多的模块，直接导入即可使用。有时候我们需要一些`个性化`(满足自己特定需求)的模块, 就可以通过自定义模块实现，即自己制作一个模块

1、自定义模块

上述提到:每个Python文件都可以作为一个模块,模块的名字就是文件的名字.也就是说将自己编写的文件导入另一个文件即可当作模块使用。

（1）自定义模块

```python
def add(x, y):
    return x + y

def minus(x, y):
    return x - y

# 模块功能测试
if __name__ == '__main__':
    res = add(10, 20)
    print(f'测试add功能:{res}')
    res = minus(3, 1)
    print(f'测试minus功能:{res}')

```

（2）使用自定义模块

```python
# 使用test01模块的内容
import test01

res = test01.add(10, 20)
print(res)

res = test01.minus(3, 1)
print(res)
```

2、注意事项

注意一:避免测试代码

在实际开发中，当一个开发人员编写完一个模块后，为了让模块能够在项目中达到想要的效果，开发人员可能会在在py文件中添加一些测试信息

此时，无论是当前文件，还是其他已经导入了该模块的文件，在运行的时候都会自动执行测试信息

解决：在 `if条件判断` 中使用

使用示例，此时只有运行 `test_03`才会执行测试代码，运行 `test_04` 时不再执行测试代码：

test01.py

```python
def add(x, y):
    return x + y


def minus(x, y):
    return x - y


# 模块功能测试
if __name__ == '__main__':
    res = add(10, 20)
    print(f'测试add功能:{res}')
    res = minus(3, 1)
    print(f'测试minus功能:{res}')
```

test02.py

```python
# 使用test01模块的内容
import test01

res = test01.add(10, 20)
print(res)

res = test01.minus(3, 1)
print(res)
```

使用解释：

```python
# 只在当前文件中运行条件才为True，导入其他文件时均为False
if __name__ == '__main__':
# __main__ 运行时程序的名称
# __name__ 系统自动赋值，不用管
# 在Run时为 __main__
# 未Run时为 文件名称
```

注意二: 模块成员同名

当导入多个模块的时候，如果模块内有同名功能，且未使用 `用法一`导入形式，后面导入的模块将会覆盖前面模块内同名的功能。

例如：

test03.py

```python
def add(x, y):
    print('test03模块下的add')
    return x + y
```

test04.py

```python
def add(a, b):
    print('test04模块下的add')
    return a + b
```

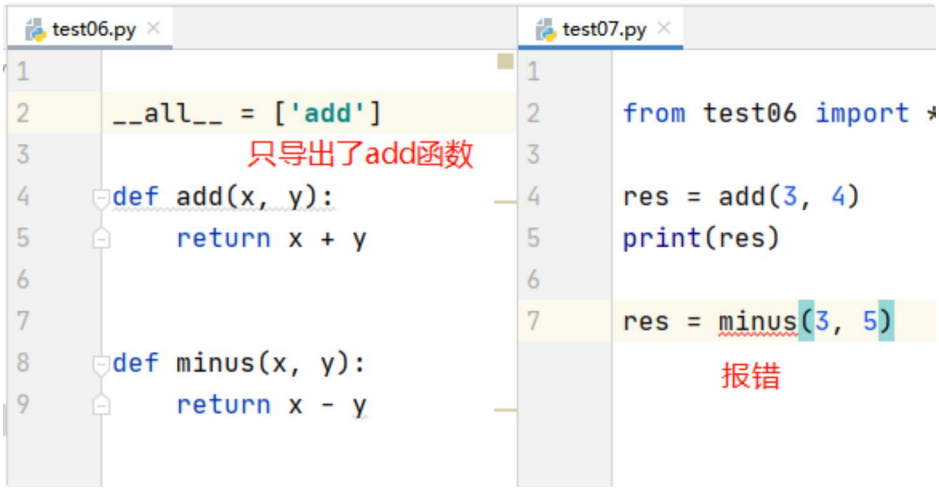
test05.py

```python
from test03 import *
from test04 import *

res = add(3, 4)
print(res)
```

注意三: 导出配置

如果一个模块文件中有 `__all__` 变量，当使用 `from xxx import *` 导入时，只能导入这个列表中的元素



- 其他导入方式不受限制

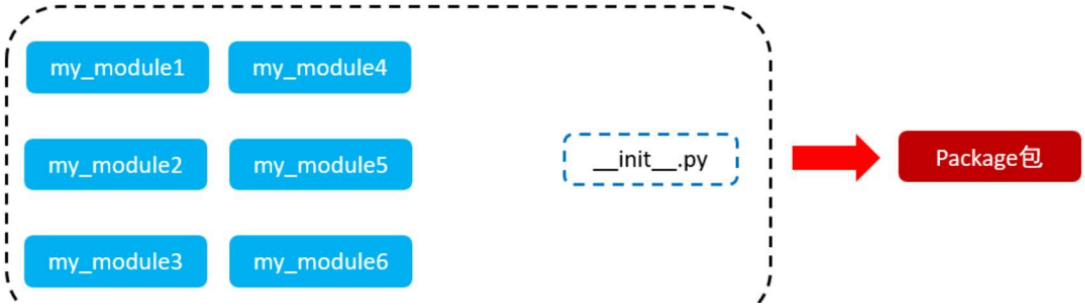
二、包

当Python的模块太多了，就可能造成一定的混乱，此时可以通过Python包的功能来管理。

1、什么是包

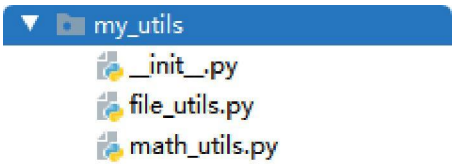
从物理上看，包就是一个文件夹，在该文件夹下包含了一个 `__init__.py` 文件，该文件夹可用于包含多个模块文件

从逻辑上看，包的本质依然是模块



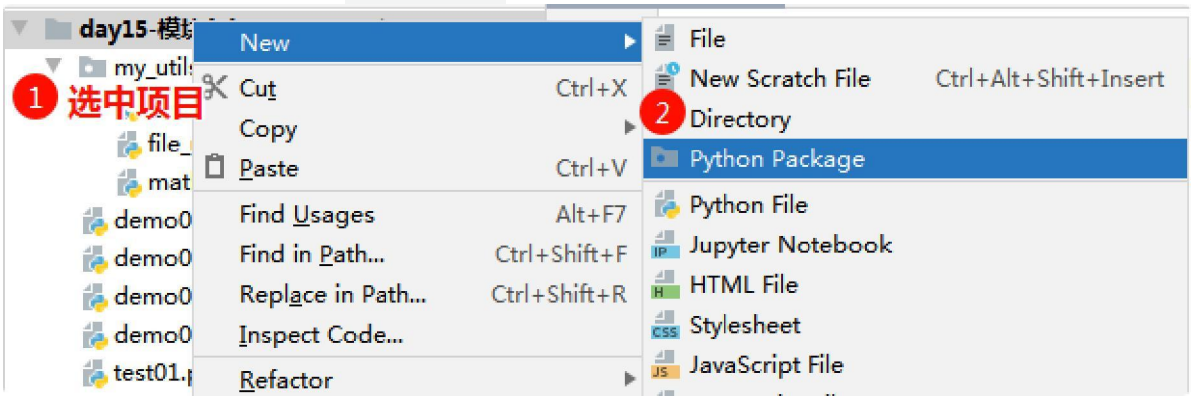
2、自定义包

当我们的模块文件越来越多时,包可以帮助我们分类管理这些模块,包的作用就是包含多个模块，但本质依然是模块



1、创建包

新建包后，包内部会自动创建 `__init__.py`文件，这个文件**控制着包的导出行为**



2、导入包

导入包与导入模块几种方式类似，例如：

```
# 导入包中模块
import 包名.模块名

# 调用包中模块的成员
包名.模块名.功能名()
```

例如：

```
import my_utils.math_utils

# 调用包中模块的成员
res = my_utils.math_utils.add(3, 4)
print(res)
```

3、限制导入

可以在 `__init__.py`文件中添加 `__all__ = ['模块名']`，控制允许导入的模块列表；

与导入模块类似 `__all__` 只针对 `from 包名 import *` 有效，而对其他方式无效

```
from my_utils import *

# 调用包中模块的成员
# res = math_utils.add(3, 4)        # 报错
# print(res)

file_utils.copy('1.png', '2.png')  # 正确
```

3、安装第三方包

第三方(其他人)开发的， Python没有内置，需要先安装才可以导入使用

在Python程序的生态中，有非常多的第三方包（非Python官方），可以极大的帮助我们提高开发效率，如：

- 科学计算中常用的： `numpy包`

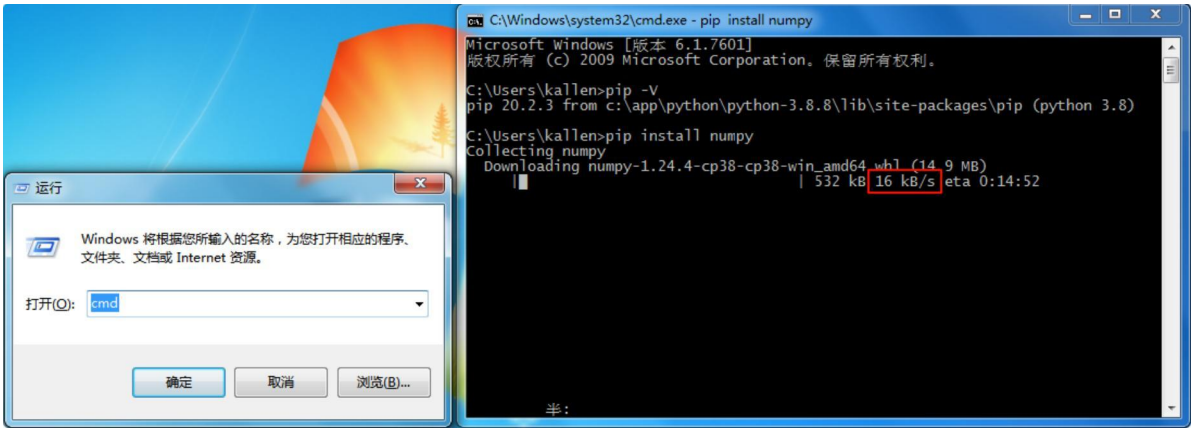
- 数据分析中常用的： pandas包
- 大数据计算中常用的： `pyspark` 、apache-flink包
- 图形可视化常用的： `matplotlib` 、 `pyecharts`
- 人工智能常用的： `tensorflow`
- ...

1、如何安装

只需要使用Python内置的pip程序即可，在终端中输入如下指令即可通过网络快速安装第三方包：

```
pip install 包名称
```

例如安装科学计算中常用的： `numpy包` ，安装成功后即可导入使用：



2、提高安装速度

由于pip是连接的国外的网站进行包的下载，下载速度经常很慢。

我们可以通过如下命令，让其连接国内的网站进行包的安装：

```
# 网站为清华大学提供的一个网站，可供pip程序下载第三方包
pip install -i https://pypi.tuna.tsinghua.edu.cn/simple 包名称
# 例如:
pip install -i https://pypi.tuna.tsinghua.edu.cn/simple numpy
```

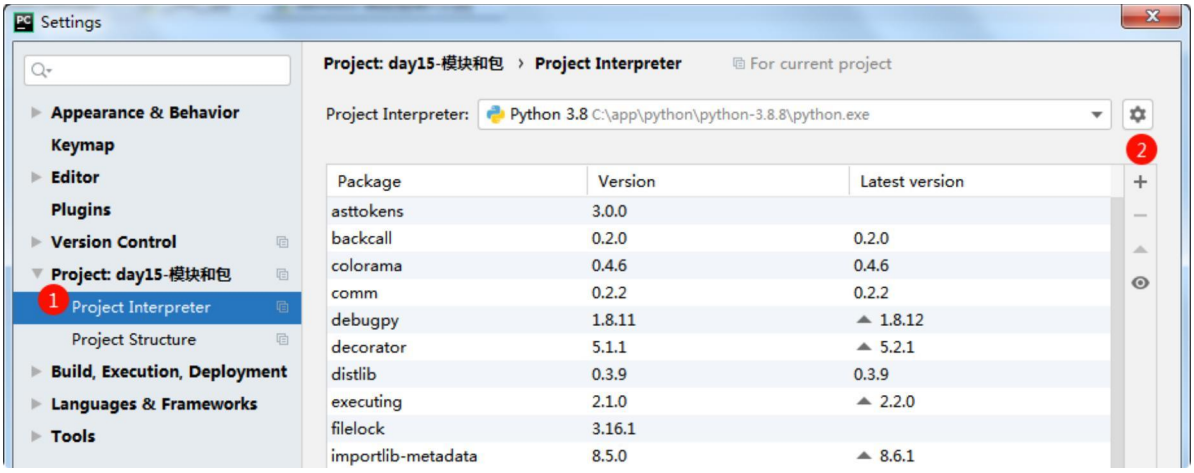
安装完成后，可以通过pip list 命令查看安装的包

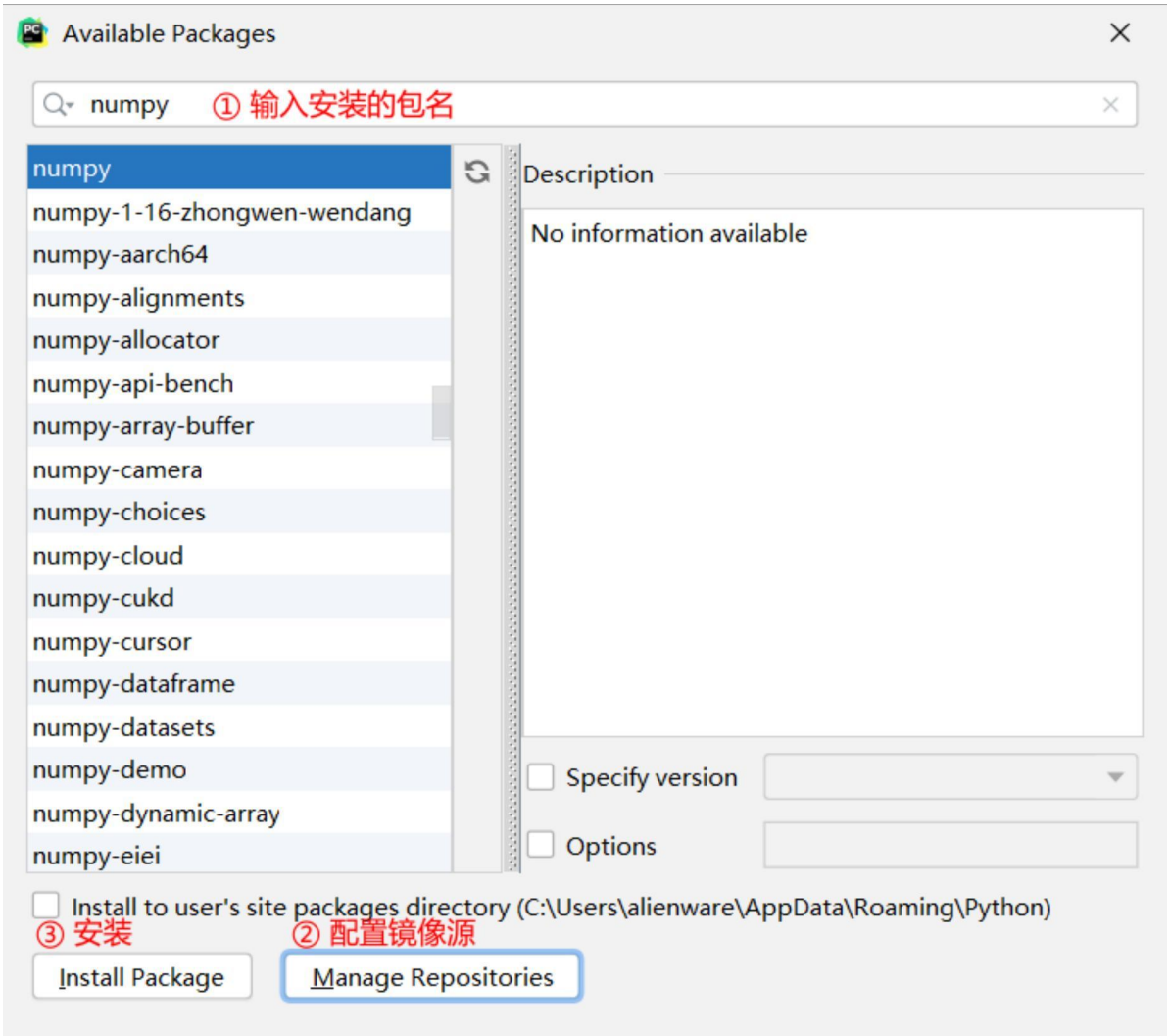
如果经常使用上述方法过于麻烦，可直接配置成镜像源之后就不需要加连接

```
python -m pip install --upgrade pip
# 升级pip版本，防止版本过低无法配置
pip config set global.index-url https://pypi.tuna.tsinghua.edu.cn/simple
# 配置为全局镜像源
```

3、在Pycharm中安装

File => Settings


Settings
Appearance & Behavior
Keymap
Editor
Plugins
Version Control
Project: day15-模块和包
1
Project Interpreter
Project Structure
Build, Execution, Deployment
Languages & Frameworks
Tools
Project: day15-模块和包 › Project Interpreter
For current project
Project Interpreter: Python 3.8 C:\app\python\python-3.8.8\python.exe
2
Package
Version
Latest version
asttokens 3.0.0
backcall 0.2.0 0.2.0
colorama 0.4.6 0.4.6
comm 0.2.2 0.2.2
debugpy 1.8.11 1.8.12
decorator 5.1.1 5.2.1
distlib 0.3.9 0.3.9
executing 2.1.0 2.2.0
filelock 3.16.1
importlib-metadata 8.5.0 8.6.1


Available Packages
numpy
① 输入安装的包名
numpy
numpy-1-16-zhongwen-wendang
numpy-aarch64
numpy-alignments
numpy-allocator
numpy-api-bench
numpy-array-buffer
numpy-camera
numpy-choices
numpy-cloud
numpy-cukd
numpy-cursor
numpy-dataframe
numpy-datasets
numpy-demo
numpy-dynamic-array
numpy-eiei
Description
No information available
Specify version
Options
Install to user's site packages directory (C:\Users\alienware\AppData\Roaming\Python)
③ 安装
② 配置镜像源
Install Package
Manage Repositories